ТЕСТОВОЕ ЗАДАНИЕ

1 задание

Предложения по улучшению данных полей:

1. **client_id:**
   - Изменить тип данных с NUMBER на INTEGER
   - явно объявить PRIMARY KEY
   - добавить автоматическую генерацию client_id с помощью - AUTO_INCREMENT

2. **client_name и client_surname:**
   - объединить в одно поле (client_name)
   - Добавить ограничение на длину, например varchar(100)

3. **login :**
   - поле login нужно дополнить ограничениями, например, уникальность UNIQUE, чтобы избежать дублирования логинов

4. **city_id:**
   - Для названий городов лучше создать отдельную таблицу для хранения информации о городах. С полями (city_id, city_name, region)
   - Изменить тип данных с NUMBER на INTEGER

5. **age :**
   - Вместо хранения данных о возрасте, лучше хранить дату рождения (birth_date DATE)
   - Изменить тип данных с NUMBER на INTEGER

6. **reg_date:**
   - В поле reg_date необходимо установить автоматическое заполнение текущей датой с помощью оператор DEFAULT вместе с функцией CURRENT_DATE

7. Все значения полей обязательны для заполнения, значит надо добавить тип NOT NULL
8. Так же можно было бы добавить отдельные поля в таблицу, а именно:
    - email VARCHAR(255) – для хранения адреса электронной почты (ограничениями уникальности UNIQUE)
    - phone VARCHAR(20) для хранения телефонного номера
    - password VARCHAR(100) – для хранения пароля учетной записи

Создание таблицы clients:

CREATE TABLE clients (

client_id INT PRIMARY KEY AUTO_INCREMENT, *-- генерируется уникальный номер id клиента и определяем первичный ключ таблицы,*

client_name VARCHAR(100) NOT NULL, *-- фамилия и имя клиента*

email VARCHAR(255) UNIQUE, *-- адрес электронной почты с ограничениями уникальности*

login VARCHAR(30) UNIQUE, *--логин, который придумал клиент с ограничениями уникальности*

password VARCHAR(100) NOT NULL, *--пароль, который придумал клиент с ограничениями уникальности*

city_id INT NOT NULL, -- *id города из таблицы cities (создается отдельно)*

birth_date DATE NOT NULL, -- *дата рождения клиента*

phone VARCHAR(20) NOT NULL, -- *телефон клиента*

reg_date DATE DEFAULT CURRENT_DATE, –*дата регистрации на сайте автоматически заполнялся текущей датой;*

FOREIGN KEY (city_id) REFERENCES cities (city_id) *-- внешний ключ для таблицы citys*

);

Создание таблицы cities:

CREATE TABLE cities(

city_id INT PRIMARY KEY,

city_name VARCHAR(100) NOT NULL,

region VARCHAR(100) NOT NULL

);

Модель базы данных

clients	
PK	client_id
	client_name
	email
	login
	password
FK	city_id
	birth_date
	phone
	reg_date

cities	
PK	city_id
	city_name
	region

2 задание

Для того чтобы обеспечить историческое хранение стоимости товара в интернет магазине, необходимо доработать архитектуру таблицы

1. ***Создадим таблицу items***, которая содержит поля с основной информацией о товаре

CREATE TABLE items (

item_id INT PRIMARY KEY AUTO_INCREMENT, --*уникальный id товара*

item_name VARCHAR(255) --*наименование товара*

);

2. ***Создадим таблицу price_change***, которая содержит данные об изменении стоимости товара

CREATE TABLE price_change (

price_id INT PRIMARY KEY AUTOINCREMENT, -- *уникальный ID записи о цене*

item_id INT NOT NULL, -- *id товара из таблицы items*

item_price DECIMAL(10, 2) NOT NULL, -- *цена товара*

start_date DATE NOT NULL, -- *дата начала действия новой цены*

end_date DATE, -- *дата окончания действия цены (если в поле -NULL, значит действует текущая цена)*

FOREIGN KEY (item_id) REFERENCES items (item_id) -- *внешний ключ для таблицы items*

);

Описание архитектуры таблицы price_change:

1. При каждом изменении цены какого либо товара, создается новая запись в таблице price_change;
2. В таблице price_change указывается новая цена товара (item_price);
3. Устанавливается дата начала действия новой цены (start_date) ;
4. Если на данный момент цена не меняется, то поле end_date остается пустым (NULL);
5. Если цена изменилась снова, то нужно заполнить поле окончание действия цены (end_date) старой цены и заполнить все необходимые поля новой цены.

Плюсы данной архитектуры:

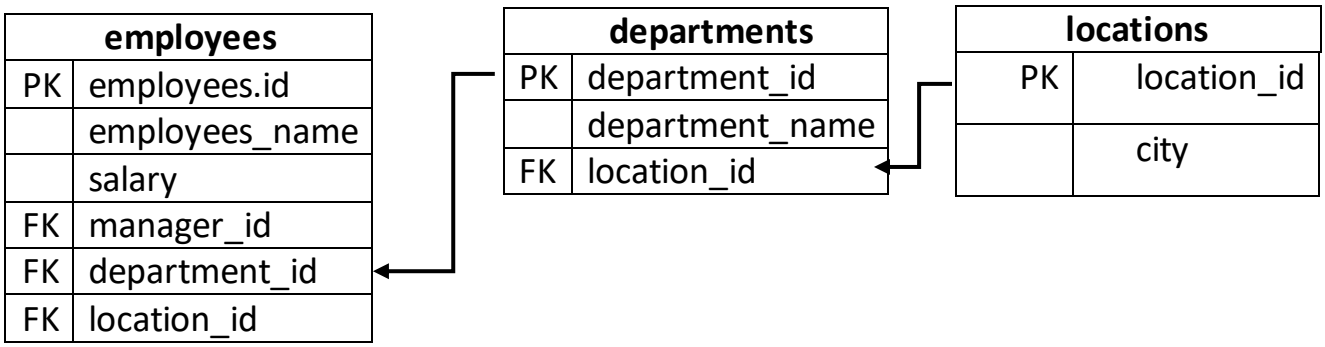
- Все изменения цен товаров сохраняются;
- Легко можно вывести текущие цены товаров (через проверку поля end_date на NULL);
- С помощью данной архитектуры можно строить различные финансовые отчеты за разные промежутки времени.

3 задание

Вывести все отделы:

- расположенные в Сеуле,
- всех сотрудников, не имеющие менеджера,
- зарабатывают в общей сложности более 100000.

Дана учебная схема HR, содержащая таблицы:



Исправленный запрос:

```
SELECT department_name
FROM locations l INNER JOIN  departments d ON l. location_id = d.location_id
                 INNER JOIN employees e ON d.department_id = e.department_id
WHERE manager_id IS NULL AND city = 'Seoul'
GROUP BY department_name
HAVING SUM(salary) > 100000;
```

В данном запросе были допущены ошибки:

1. В условии задания требовалось вывести только отделы (department_name);
2. Соединение таблиц необходимо осуществлять через INNER JOIN;
3. В соединение таблиц необходимо добавить третью таблицу locations;
4. Для проверки на NULL нужно использовать IS NULL;
5. Не нужно использовать подзапрос для фильтрации по городу 'Seoul', а использовать условие (city = 'Seoul') напрямую в запросе;
6. Условие для вычисления суммы зарплат SUM(salary) > 100000, нужно использовать после группировки GROUP BY с помощью оператора HAVING;
7. Оператор >= заменили на >.

4 задание

Причин по которым, я не могу перепрыгнуть через карандаш может быть несколько:

1. Карандаш закатился в труднодоступное место (например, под диван);
2. Карандаш упал, совершенно в другом месте, в отличном от моего места прибывания (например, в другом городе, стране, планете и т.д.);
3. И самая простая причина, по которой я не могу перепрыгнуть, это потому, что я сейчас читаю данное задание и прыгать в этот момент не могу;
4. Карандаш слишком маленький, чтобы через него прыгать, можно просто перешагнуть.

P.S.

Спасибо за интересное задание. Буду рада обратной связи и возможности развиваться в направлении Data Engineer.